深入浅出 LoRA

by 知乎 https://zhuanlan.zhihu.com/p/650197598

论文：LoRA: Low-rank Adaptation of Large Language Models

一、什么是 LoRA

1.1 针对的问题

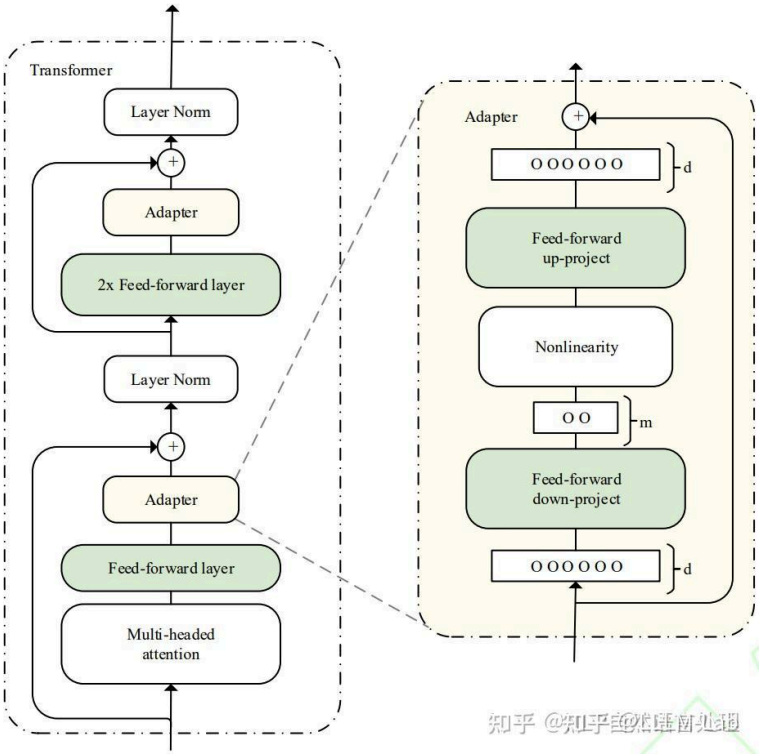
全量参数 Fine-tune 需要调整模型全部参数，随着预训练模型规模的不断扩大（GPT-3，175B），全量 Fine-tune 的资源压力也倍增。高效、快速对模型进行领域或任务的微调，在大模型时代极其重要。

1.2 替代解决方案

针对全量 Fine-tune 的昂贵问题，目前主要有两种解决方案：

Adapt Tuning。即在模型中添加 Adapter 层，在微调时冻结原参数，仅更新 Adapter 层。

具体而言，其在预训练模型每层中插入用于下游任务的参数，即 Adapter 模块，在微调时冻结模型主体，仅训练特定于任务的参数。



每个 Adapter 模块由两个前馈子层组成，第一个前馈子层将 Transformer 块的输出作为输入，将原始输入维度 d 投影到 m，通过控制 m 的大小来限制 Adapter 模块的参数量，通常情况下 m << d。在输出阶段，通过第二个前馈子层还原输入维度，将 m 重新投影到 d，作为 Adapter 模块的输出(如上图右侧结构)。

LoRA 事实上就是一种改进的 Adapt Tuning 方法。但 Adapt Tuning 方法存在推理延迟问题，由于增加了额外参数和额外计算量，导致微调之后的模型计算速度相较原预训练模型更慢。

Prefix Tuning。该种方法固定预训练 LM，为 LM 添加可训练，任务特定的前缀，这样就可以为不同任务保存不同的前缀，微调成本也小。具体而言，在每一个输入 token 前构造一段与下游任务相关的 virtual tokens 作为 prefix，在微调时只更新 prefix 部分的参数，而其他参数冻结不变。

也是目前常用的微量微调方法的 Ptuning，其实就是 Prefix Tuning 的一种改进。但 Prefix Tuning 也存在固定的缺陷：模型可用序列长度减少。由于加入了 virtual tokens，占用了可用序列长度，因此越高的微调质量，模型可用序列长度就越低。

1.3 LoRA 的思路

如果一个大模型是将数据映射到高维空间进行处理，这里假定在处理一个细分的小任务时，是不需要那么复杂的大模型的，可能只需要在某个子空间范围内就可以解决，那么也就不需要对全量参数进行优化了，我们可以定义当对某个子空间参数进行优化时，能够达到全量参数优化的性能的一定水平（如90%精度）时，那么这个子空间参数矩阵的秩就可以称为对应当前待解决问题的**本征秩（intrinsic rank）**。

> - 在数学上，“秩”（Rank）可以简单理解为矩阵的“复杂度”或“信息量”。满秩就是复杂度最高。
> - **本征秩**的意思是：解决这个特定任务，**真正需要的最低复杂度是多少**。
> - 举例：如果我只需要更新 0.1% 的参数（低秩），就能达到更新 100% 参数（全量参数）90% 的效果，那么这个任务的“本征秩”就很低。

预训练模型本身就隐式地降低了本征秩，当针对特定任务进行微调后，模型中权重矩阵其实具有更低的本征秩（intrinsic rank）。同时，越简单的下游任务，对应的本征秩越低。（Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning）因此，权重更新的那部分参数矩阵尽管随机投影到较小的子空间，仍然可以有效的学习，可以理解为针对特定的下游任务这些权重矩阵就不要求满秩。我们可以通过优化密集层在适应过程中变化的秩分解矩阵来间接训练神经网络中的一些密集层，从而实现仅优化密集层的秩分解矩阵来达到微调效果。

> - **预训练已经做了大部分工作：** 大模型在预训练阶段（看遍了互联网数据）已经学会了语言的基本规律。这时候，模型的权重矩阵虽然很大，但其中包含了很多冗余信息，本质上它的“有效秩”并没有看起来那么高。
> - **微调只是“微调”：** 当你让模型去学一个新任务（比如学医），你不需要重新教它说话（预训练已会），只需要教它医学知识。这个“增量”非常小，所以对应的**本征秩非常低**。
> - **任务越简单，秩越低：** 比如只是让模型学会把“你好”翻译成“Hello”，这太简单了，可能只需要极低的秩（极少的参数变化）就能学会。
> - **LoRA 的做法：** 既然权重更新只需要在一个很小的低秩子空间里进行，那我们就不去动原来的大矩阵，而是**在旁边加两个小矩阵（秩分解矩阵）**，只训练这两个小矩阵。这就是 LoRA 的核心。

例如，假设预训练参数为 θ_0^D，在特定下游任务上密集层权重参数矩阵对应的本征秩为 θ^d，对应特定下游任务微调参数为 θ^D，那么有：

$$\theta^D = \theta_0^D + \theta^d M$$

这个 M 即为 LoRA 优化的秩分解矩阵。

> **【微调后的新模型】 = 【原来的老模型】 + 【一点点小改动】**

> 你可能会问：*“为什么要写成两个东西相乘（θd×Mθd×M）？*
>
> **这就是 LoRA 的精髓：低秩分解（Low-Rank Decomposition）。**
>
> **举个数字例子：**
>
> 假设大模型里有一个巨大的权重矩阵，大小是 **1000×1000**。
>
> - 如果要全量微调，你需要更新 **100万** 个参数（1000×1000）。这太贵了，显存不够。
>
> LoRA 认为，其实这个矩阵的变化没那么复杂，可以用两个**很瘦长/很扁**的小矩阵相乘来代替：
>
> - θ^d**(矩阵 B)：** 大小设为 1000×8 （秩 r=8）。
> - M **(矩阵 A)：** 大小设为 8×1000 （秩 r=8）。
>
> 当你把它们乘起来 ($\theta^d \times M$)：
>
> - 结果依然是一个 1000×1000 的大矩阵（和原模型一样大，可以相加）。

> - **但是！** 你需要训练的参数量只有：(1000×8)+(8×1000)=8000+8000=**1.6万**。
>
> **结论：**
> 公式里的 $\theta^d M$ 就是在说：**“我们要更新的这部分参数，是由两个极小的矩阵乘出来的。”** 这就是为什么 LoRA 训练起来那么快、那么省显存的原因。

1.4 LoRA 的优势

1. 可以针对不同的下游任务构建小型 LoRA 模块，从而在共享预训练模型参数基础上有效地切换下游任务。
2. LoRA 使用自适应优化器（Adaptive Optimizer），不需要计算梯度或维护大多数参数的优化器状态，训练更有效、硬件门槛更低。

> - **显存杀手是谁？** 大模型训练最吃显存的往往不是模型参数本身，而是**优化器状态**。以常用的 Adam/AdamW 为例，它需要为**每个可训练参数**额外保存一阶动量（momentum）和二阶动量（variance），通常占模型参数量的 **2~3 倍**。
> - **全量微调**：13B 模型 ≈ 26GB 参数，Adam 状态就要 50~80GB 显存，必须多卡 A100。
> - **LoRA 做法**：冻结 99% 以上的基座参数，**只有 LoRA 的那两个小矩阵需要梯度和优化器状态**。显存占用断崖式下降，单张消费级显卡（如 RTX 3090/4090 24GB）就能流畅微调 7B~13B 模型。
> - 💡 **注意原文表述微调**：LoRA 本身不是优化器，而是**让自适应优化器（如 AdamW）只需要维护极少参数的状态**，从而避开显存瓶颈。

3. LoRA 使用简单的线性设计，在部署时将可训练矩阵与冻结权重合并，不存在推理延迟。

> 数学本质：LoRA 的更新公式是加性的：
>
> $$W_{\text{new}} = W_{\text{frozen}} + \Delta W = W_{\text{frozen}} + B \times A$$
>
> 部署技巧：推理阶段，可以直接把 $B \times A$ 离线加到 W_{frozen} 上，得到一个新的完整权重矩阵 W_{merged}。部署时加载 W_{merged}，模型结构、计算图、FLOPs 与原模型完全一致。
>
> 对比其他 PEFT 方法：
> Adapter / Prefix-Tuning / Prompt Tuning 会在前向传播中额外增加网络层或 token，推理时必然增加计算量和延迟。
> LoRA 是唯一能做到“训练时参数高效，推理时零额外开销”的主流微调方法。

4. LoRA 与其他方法正交，可以组合。

> - **正交"在 ML 中的含义**：指两种技术作用在不同维度，互不冲突、可叠加使用。
> - **LoRA 只改变"权重更新机制"**，不改动模型架构、数据流、损失函数或训练范式。因此它可以像乐高一样与其他技术自由组合：
> - 🔹 **+ 量化** → **QLoRA**（4bit/8bit 量化基座 + LoRA，显存再降 70%，目前最流行组合）
> - 🔹 **+ 提示词工程** → Prompt + LoRA 双管齐下
> - 🔹 **+ 其他微调策略** → 部分层用 LoRA，部分层全量微调；或 LoRA + Dropout / Weight Decay 等正则化
> - 🔹 **+ 分布式训练** → 与 DeepSpeed、FSDP 完全兼容

二、LoRA 的原理

2.1 低秩参数化更新矩阵

LoRA 假设权重更新的过程中也有一个较低的本征秩，对于预训练的权重参数矩阵 $W_0 \in \mathbb{R}^{d\times k}$（$d$ 为上一层输出维度，k 为下一层输入维度），使用低秩分解来表示其更新：

$$W_0 + \Delta W = W_0 + BA \quad \text{where } B \in \mathbb{R}^{d\times r}, A \in \mathbb{R}^{r\times k}$$

在训练过程中，W_0 冻结不更新，A、B 包含可训练参数。

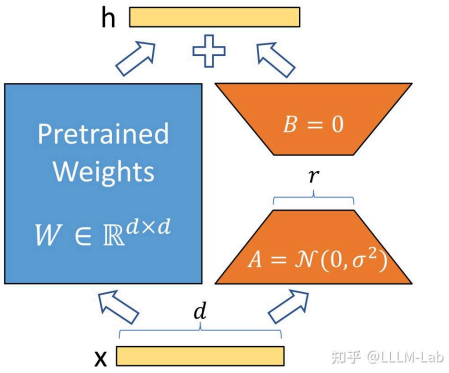
因此，LoRA 的前向传递函数为：

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

在开始训练时，对 A 使用随机高斯初始化，对 B 使用零初始化，然后使用 Adam 进行优化。训练思路如图：

> 💡 **设计哲学：**
>
> 1. **保持预训练知识**：从"什么都不改"开始，确保初始性能不退化
> 2. **对称性破缺**：如果 A 也初始化为 0，那么 A 和 B 的梯度都会是 0，**永远学不起来**（对称性问题）。所以 A 必须随机初始化。
> 3. **稳定训练**：从零更新开始，逐步学习任务特定的调整，避免剧烈变化导致训练不稳定。

> **对比其他初始化：**
>
> - ❌ 如果 B 也随机初始化：初始 ΔW 是随机噪声，可能破坏预训练知识
> - ❌ 如果 A 也零初始化：梯度消失，无法学习
> - ✅ A **随机 +** B **零**：完美平衡"保持原样"和"可学习"



2.2 应用于 Transformer

在 Transformer 结构中，LoRA 技术主要应用在注意力模块的四个权重矩阵：W_q、W_k、W_v、W_o，而冻结 MLP（多层感知机，即前馈神经网络部分） 的权重矩阵。

通过消融实验发现同时调整 W_q 和 W_v 会产生最佳结果。

> - **直观理解：** W_q 决定了模型"关注哪里"，W_v 决定了模型"提取什么信息"。这两个对特定任务的适配最关键。而 W_k 和 W_o 相对次要。

在上述条件下，可训练参数个数为：

$$\Theta = 2 \times L_{LoRA} \times d_{model} \times r$$

其中，L_{LoRA} 为应用 LoRA 的权重矩阵的个数，d_{model} 为 Transformer 的输入输出维度，r 为设定的 LoRA 秩。

一般情况下，r 取到 4、8、16。

> - r **(Rank)** 是 LoRA 的超参数，代表"低秩"的秩。
> - r **越小**：参数量越少，训练越快，但可能学不到复杂的任务特征（欠拟合）。
> - r **越大**：参数量越多，表达能力越强，越接近全量微调，但显存占用增加。
> - **经验法则**：
> - 简单任务（如情感分类）：$r = 4$ 或 8 就够了。

> - 复杂任务（如指令微调、代码生成）：$r = 16$ 或 64。
> - 通常 $r = 8$ 是一个万能的默认值。

三、代码实现

目前一般通过 peft 库来实现模型的 LoRA 微调。peft 库是 huggingface 开发的第三方库，其中封装了包括 LoRA、Adapt Tuning、P-tuning 等多种高效微调方法，可以基于此便捷地实现模型的 LoRA 微调。

本文简单解析 peft 库中的 LoRA 微调代码，简单分析 LoRA 微调的代码实现。

3.1 实现流程

LoRA 微调的内部实现流程主要包括以下几个步骤：

1. 确定要使用 LoRA 的层。peft 库目前支持调用 LoRA 的层包括：nn.Linear、nn.Embedding、nn.Conv2d 三种。

```
from peft import LoraConfig, TaskType

# 配置 LoRA
config = LoraConfig(
    r=8,                            # 秩
    lora_alpha=16,                  # 缩放系数, scaling = alp
ha/r ,
    # 一般设置为r的两倍。控制LoRA更新的幅度，让BA输出不要太大/太小。
相当于归一化
    # 换 r 时，其他超参可以不动，且训练稳定性差不多。
    target_modules=["q_proj", "v_proj"],  # 🔑 关键：指定哪些
层用 LoRA
    lora_dropout=0.1,               # LoRA 矩阵的 dropout 概
率，防止 LoRA 过拟合
    bias="none",                    # 是否训练偏置项
    task_type=TaskType.CAUSAL_LM    # 任务类型：因果语言模型。
    # 比如：CAUSAL_LM = decoder-only 大模型（LLaMA、Qwen、Llam
a 3）;
    # SEQ_2_SEQ_LM = encoder-decoder（T5）;
    # TOKEN_CLS = 分类任务
)
```

> 为什么要缩放？因为：
>
> - **低秩矩阵 BA 的值通常很小**
> - 如果不缩放，LoRA 学习不动
> - 缩放太大 → 模型震荡、不收敛
> - 缩放太小 → 学习太慢、效果差
>
> 所以 `alpha` 就是用来 **调节 LoRA 更新强度** 的旋钮。

2. 对每一个要使用 LoRA 的层，替换为 LoRA 层。所谓 LoRA 层，实则是在该层原结果基础上增加了一个旁路，通过低秩分解（即矩阵 A 和矩阵 B）来模拟参数更新。

> **底层原理**：
>
> - PEFT 会遍历模型的 `named_modules()`，查找名字匹配 `target_modules` 的模块。
> - 只支持 `nn.Linear` / `nn.Embedding` / `nn.Conv2d`，因为：
> - 这三类是**参数矩阵型**操作，适合做低秩分解 $W + BA$。
> - LayerNorm、激活函数等没有权重矩阵，无法应用 LoRA。

3. 冻结原参数，进行微调，更新 LoRA 层参数。

3.2 确定 LoRA 层

在进行 LoRA 微调时，首先需要确定 LoRA 微调参数，其中一个重要参数即是 target_modules。target_modules 一般是一个字符串列表，每一个字符串是需要进行 LoRA 的层名称，例如：

```
target_modules = ["q_proj","v_proj"]
```

这里的 **`q_proj`** 即为注意力机制中的 W_q，**`v_proj`** 即为注意力机制中的 W_v。我们可以根据模型架构和任务要求自定义需要进行 LoRA 操作的层。

在创建 LoRA 模型时，会获取该参数，然后在原模型中找到对应的层，该操作主要通过使用 **`re`** 对层名进行正则匹配实现：

```
# 找到模型的各个组件中，名字里带"q_proj"，"v_proj"的
target_module_found = re.fullmatch(self.peft_config.target_
```

```
modules, key)
# 这里的 key，是模型的组件名
```

3.3 替换 LoRA 层

对于找到的每一个目标层，会创建一个新的 LoRA 层进行替换。

LoRA 层在具体实现上，是定义了一个基于 Lora 基类的 Linear 类，该类同时继承了 nn.Linear 和 LoraLayer。LoraLayer 即是 Lora 基类，其主要构造了 LoRA 的各种超参：

父类	作用	带来的能力
`nn.Linear`	PyTorch 原生线性层	✅ 直接复用 $W_0x + b$ 的前向计算✅ 兼容所有基于 `nn.Linear` 的模型代码
`LoraLayer`	LoRA 自定义基类	✅ 统一管理 LoRA 超参（r, α, dropout）✅ 提供权重合并/解合并的逻辑

设计哲学：

> "Is-a" + "Has-a" 的结合
>
> - LoRA 层 **是一个** 线性层（所以继承 `nn.Linear`）
> - LoRA 层 **拥有** LoRA 的特性（所以继承 `LoraLayer`）

把低秩矩阵 BA 的输出，缩放到一个稳定、合理、符合原模型分布的数值范围 → 这就叫 **归一化（Normalization）**

- **归一化（Normalization）**：把数据缩放到 **固定区间**，通常是 **[0,1]** 或 **[-1,1]**
- **标准化（Standardization）**：把数据变成 **均值 0，方差 1** 的标准正态分布

```
class LoraLayer:          # "超参管理中心"
    def __init__(
        self,
        r: int, # LoRA 的秩
        lora_alpha: int, # 归一化参数
        lora_dropout: float, # LoRA 层的 dropout 比例
        merge_weights: bool, # eval 模式中，是否将 LoRA 矩阵的
值加到原权重矩阵上
    ):
        self.r = r
        self.lora_alpha = lora_alpha
```

```
        # Optional dropout
        if lora_dropout > 0.0:
            self.lora_dropout = nn.Dropout(p=lora_dropout)
        else:
            self.lora_dropout = lambda x: x
        # Mark the weight as unmerged
        self.merged = False       # 标记：权重是否已合并（用于推理优化）
        self.merge_weights = merge_weights    # 配置：是否允许自动合并
        self.disable_adapters = False    # 关：是否临时禁用 LoRA（用于 A/B 测试）
```

nn.Linear 就是 Pytorch 的线性层实现。Linear 类就是具体的 LoRA 层，其主要实现如下：

```
class Linear(nn.Linear, LoraLayer):        # 真正的 LoRA 实现
    # LoRA 层
    def __init__(
        self,
        in_features: int,
        out_features: int,
        r: int = 0,
        lora_alpha: int = 1,
        lora_dropout: float = 0.0,
        fan_in_fan_out: bool = False,
        merge_weights: bool = True,
        **kwargs,
    ):
        # 继承两个基类的构造函数
        nn.Linear.__init__(self, in_features, out_features, **kwargs)
        LoraLayer.__init__(self, r=r, lora_alpha=lora_alpha,
                                lora_dropout=lora_dropout, merge_weights=merge_weights)
```

```
        self.fan_in_fan_out = fan_in_fan_out
        # Actual trainable parameters
        if r > 0:
            # 参数矩阵 A
            self.lora_A = nn.Linear(in_features, r, bias=Fa
lse)
            # 参数矩阵 B
            self.lora_B = nn.Linear(r, out_features, bias=F
alse)
            # 归一化系数
            self.scaling = self.lora_alpha / self.r
            # 冻结原参数，仅更新 A 和 B
            self.weight.requires_grad = False
        # 初始化 A 和 B
        self.reset_parameters()
        # fan_in_fan_out：适配不同框架的权重存储习惯（见下文详解）
        # 这是一个历史兼容的坑
        if fan_in_fan_out:
            self.weight.data = self.weight.data.T
```

替换时，直接将原层的 weight 和 bias 复制给新的 LoRA 层，再将新的 LoRA 层分配到指定设备即可。

3.4 训练

实现了 LoRA 层的替换后，进行微调训练即可。由于在 LoRA 层中已冻结原参数，在训练中只有 A 和 B 的参数会被更新，从而实现了高效微调。训练的整体过程与原 Fine-tune 类似，此处不再赘述。由于采用了 LoRA 方式，forward 函数也会对应调整：

```
def forward(self, x: torch.Tensor):
                # ┌─ 分支 1：禁用适配器 + 已合并权重 → 需要先"解
合并"
        if self.disable_adapters:
            if self.r > 0 and self.merged:
                    # 解合并：W₀ = W_merged - α/r·BA
                self.weight.data -= (
                    transpose(self.lora_B.weight @ self.lor
a_A.weight,
```

```
                                        self.fan_in_fan_out) *
self.scaling
                )
                self.merged = False

            return F.linear(x, transpose(self.weight, self.
fan_in_fan_out),
                                            bias=self.bias)
        '''主要分支'''
          # ┌─ 分支 2（主分支）：启用适配器 + 未合并 → 正常双路径计
算 ⭐ 重点看这里
        elif self.r > 0 and not self.merged:
            result = F.linear(x, transpose(self.weight,
                                self.fan_in_fan_out), bias=
self.bias)
            if self.r > 0:
                result += self.lora_B(self.lora_A(self.lora
_dropout(x)))
                                        * self.scaling
            return result
        # ┌─ 分支 3：r=0 或 已合并权重 → 退化回普通 Linear
        else:
            return F.linear(x, transpose(self.weight,
                                self.fan_in_fan_out), bias=
self.bias)
```

这段代码完美实现了我们熟悉的 LoRA 公式：

$$h = \underbrace{W_0 x + b}_{\text{原始路径}} + \underbrace{\frac{\alpha}{r} \cdot B\left(A\left(\text{dropout}(x)\right)\right)}_{\text{LoRA 旁路}}$$

上述代码由于考虑到参数合并问题，有几个分支，此处我们仅阅读第二个分支即 elif 分支即可。基于 LoRA 的前向计算过程如前文公式所示，首先计算原参数与输入的乘积，再加上 A、B 分别与输入的乘积即可。

四、使用 peft 实现大模型微调

peft （Parameter-Efficient Fine-Tuning，参数高效微调）进行了很好的封装，支持我们便捷、高效地对大模型进行微调。此处以开源大模型 ChatGLM2-6B 为例，简要

介绍如何使用 peft 对大模型进行微调。此处我们假设数据集已处理完成，不再介绍数据处理过程。

加载所需使用库：

```
import torch.nn as nn
from transformers import AutoTokenizer, AutoModel
from peft import get_peft_model, LoraConfig, TaskType, PeftModel
from transformers import Trainer
```

首先需要加载原模型与原 tokenizer，此处我们使用 transformers 进行加载：

```
# 加载底座模型
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH, trust_remote_code=True)
model = AutoModel.from_pretrained(
    MODEL_PATH, load_in_8bit=False, trust_remote_code=True, device_map="auto"
)
# MODEL_PATH: 模型所在路径（本地文件夹 或 Hugging Face 地址）
# load_in_8bit: =False: 用FP16/FP32 全精度加载模型;
                              # =True: 用 8bit 量化加载，省显存，但速度慢一点
# trust_remote_code: 允许执行模型文件夹里的自定义代码（模型结构定义）
# device_map="auto": 自动分配模型到 GPU/CPU

# 对底座模型做一些设置
model.gradient_checkpointing_enable()
model.enable_input_require_grads()
model.is_parallelizable = True
model.model_parallel = True
model.config.use_cache = (
    False  # silence the warnings. Please re-enable for inference!
)
```

> 1️⃣ `model.gradient_checkpointing_enable()` | **梯度检查点**

> 🔍 **核心痛点：显存被“激活值”吃光了**
>
> 训练时，PyTorch 默认会保存**每一层的中间输出（激活值）**，因为反向传播计算梯度时需要用到它们。
>
> - 假设模型有 30 层，每层输出是 `(batch, seq_len, hidden)`。
> - 显存占用 ≈ `30 × 每层激活值大小`。
> - 对 6B/7B 模型，仅激活值就能占 15~20GB，直接 OOM（显存溢出）。
>
> ⚙️ **底层原理：用时间换空间**
>
> 开启后，PyTorch **不再保存所有层的激活值**，而是：
>
> 1. **前向传播**：只保存少数关键节点的激活值（如每隔几层存一次，或只存输入）。
> 2. **反向传播**：需要某层梯度时，**临时重新跑一遍该层的前向计算**，算出激活值后立即用于求导，然后丢弃。
>
> 💡 **通俗比喻**
>
> > 就像打游戏存档。**不开**：每走一步都自动存档（占硬盘，读档快）。**开启**：只在 checkpoint 存档（省硬盘），死后从上一个 checkpoint 重新打一遍（费时间，但能玩下去）。

> 2️⃣ `model.enable_input_require_grads()` | **允许输入计算梯度**
>
> 🔍 **核心痛点：检查点导致梯度图断裂**
>
> `gradient_checkpointing` 底层依赖 `torch.utils.checkpoint.checkpoint`，它是一个 `torch.autograd.Function`。**PyTorch 的硬性要求**：传入 checkpoint 函数的张量，必须 `requires_grad=True`，否则反向传播时无法构建计算图。
>
> ⚙️ **底层原理**
>
> 这个方法会遍历模型的 Embedding 层和输入相关模块，强制打开梯度开关：

> 3️⃣ `is_parallelizable = True` / `model.model_parallel = True` | **并行声明**
>
> 🔍 **核心痛点：HF Trainer 的安全拦截**
>
> HuggingFace `Trainer` 和 `accelerate` 在启动分布式训练前，会做**合法性检查**。如果发现模型没声明自己支持并行，会抛出警告甚至中断。
>
> ⚙️ **底层原理**
>
> 这两个属性**纯粹是元数据开关（Metadata Flags）**，不改变任何计算逻辑：
>
> - `is_parallelizable = True`：告诉 `accelerate`：“我的参数可以安全切分到多卡，别拦我。”

> - `model.model_parallel = True`：历史遗留标志，兼容旧版 HF 的 Pipeline Parallelism 检查。
>
> ⚠️ **重要提醒**：设为 `True` **不会自动实现并行**！它只是“放行许可证”。真正的多卡切分靠的是：
>
> - `device_map="auto"`（自动层间切分）
> - `accelerate` / `DeepSpeed` / `FSDP`（分布式策略）

> 4️⃣ `model.config.use_cache = False` | **关闭 KV Cache**
>
> 🔍 **核心痛点：训练时 KV Cache 会破坏计算图**
>
> KV Cache 是 **推理加速技术**：生成第 t*t* 个 token 时，复用前 t−1*t*−1 个 token 的 Key/Value，避免重复计算 Attention。
>
> 但在**训练阶段**：
>
> 1. **梯度断裂**：Cache 是 detached 的（不记录梯度），反向传播时梯度无法流过历史 token，模型学不到上下文依赖。
> 2. **显存泄漏**：Cache 会随序列长度线性增长，训练长序列时直接 OOM。
> 3. **逻辑冲突**：训练是“批量并行计算整个序列”，不需要自回归缓存。
>
> ⚙️ **底层原理**
>
> `model.config.use_cache` 是 `transformers` 的全局配置。设为 `False` 后：
>
> - 模型 `forward()` 会跳过 `past_key_values` 的读取与返回。
> - Attention 层每次从头计算 *QKT*，保证完整的计算图。
>
> QKT
>
> - 训练结束后，推理时改回 `True` 即可恢复加速。

> 参数量 × 精度所占字节 ≈ 模型基础显存
>
> - FP32：6B × 4 = **24GB**
> - FP16/BF16：6B × 2 = **12GB**
> - INT8：6B × 1 = **6GB**
> - INT4：6B × 0.5 = **3GB**

接着，设定 peft 参数：

```
peft_config = LoraConfig(
            task_type=TaskType.CAUSAL_LM,
             # 是否推理模式。训练必须 False，否则不会创建可训练参数
            inference_mode=False,
            r=8,
            lora_alpha=32,
            lora_dropout=0.1,
        )
```

注意，对不同的模型，LoRA 参数可能有所区别。例如，对于 ChatGLM，无需指定 target_modeules，peft 可以自行找到；对于 BaiChuan，就需要手动指定。task_type 是模型的任务类型，大模型一般都是 CAUSAL_LM 即传统语言模型。

然后获取 LoRA 模型：

```
model = get_peft_model(model, peft_config)
```

此处的 get_peft_model 的底层操作，即为上文分析的具体实现。

最后使用 transformers 提供的 Trainer 进行训练即可：

```
trainer = Trainer(
            model=model,
            train_dataset=dataset,
            args=training_args,
            data_collator=lambda x : data_collator_glm(x, t
okenizer),
        )
trainer.train()
```

> data_collator：将样本打包成 batch。ChatGLM 需要特殊处理（如填充 `[gMASK]`、生成 `position_ids`），不能用默认 collator

> 💡 **为什么不需要手动过滤参数？**
> `Trainer` 底层调用 `model.parameters()` 时，PyTorch 会自动忽略 `requires_grad=False` 的张量。`peft` 已经冻结了底座，所以优化器天然只更新 LoRA 参数。

限于篇幅，此处仅简析使用 peft 进行 LoRA 微调的大致流程。

补充

维度	训练 (Training)	推理 (Inference)
核心目标	让模型"学会"知识（更新参数）	让模型"使用"知识（输出结果）
是否需要梯度	✅ 必须计算并反向传播	❌ 不需要，可关闭 `torch.no_grad()`
参数是否更新	✅ 优化器会更新权重	❌ 权重完全冻结
Dropout	✅ 开启，防止过拟合	❌ 关闭，保证输出确定性
BatchNorm	✅ 用当前 batch 统计量 + 更新全局统计量	❌ 用训练时累积的全局统计量
KV Cache	❌ 关闭（会破坏梯度）	✅ 开启（加速自回归生成）
显存占用	🔴 极高（存激活值+梯度+优化器状态）	🟢 较低（只存权重+少量中间结果）
计算量	🔴 前向+反向 ≈ 2~3倍前向	🟢 仅前向传播
延迟关注	关注**吞吐量**（每秒处理多少样本）	关注**首字延迟** + **生成速度**（用户感知）
批处理策略	大 batch 提高吞吐，可填充对齐	小 batch 或单样本，减少等待延迟
精度要求	可用混合精度（FP16/BF16）加速	有时需 FP32 保证数值稳定（如长文本生成）